

230905290 Week 7 PPL Lab

Rohan Shenoy 32 CSE D

Q1] Write a CUDA program to count the number of times a given word is repeated in a sentence (use atomic function)

CODE]

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

__global__ void countWordKernel(const char* sentence, int sentenceLen,
const char* target, int targetLen, int* totalCount) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx <= (sentenceLen - targetLen)) {
        bool match = true;

        for (int i = 0; i < targetLen; i++) {
            if (sentence[idx + i] != target[i]) {
                match = false;
                break;
            }
        }

        if (match) {
            bool prefixOk = (idx == 0) || (sentence[idx - 1] == ' ');
            bool suffixOk = (idx + targetLen == sentenceLen) ||
(sentence[idx + targetLen] == ' ');

            if (prefixOk && suffixOk) {
                atomicAdd(totalCount, 1);
            }
        }
    }
}
```

```

int main() {
    const char* h_sentence = "the quick brown fox jumps over the lazy dog
the";
    const char* h_target = "the";

    int sentenceLen = strlen(h_sentence);
    int targetLen = strlen(h_target);

    char *d_sentence, *d_target;
    int *d_count;
    int h_count = 0;

    cudaMalloc((void**)&d_sentence, sentenceLen + 1);
    cudaMalloc((void**)&d_target, targetLen + 1);
    cudaMalloc((void**)&d_count, sizeof(int));

    cudaMemcpy(d_sentence, h_sentence, sentenceLen + 1,
cudaMemcpyHostToDevice);
    cudaMemcpy(d_target, h_target, targetLen + 1, cudaMemcpyHostToDevice);
    cudaMemcpy(d_count, &h_count, sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (sentenceLen + threadsPerBlock - 1) /
threadsPerBlock;

    countWordKernel<<<blocksPerGrid, threadsPerBlock>>>(d_sentence,
sentenceLen, d_target, targetLen, d_count);

    cudaMemcpy(&h_count, d_count, sizeof(int), cudaMemcpyDeviceToHost);

    printf("The word '%s' appeared %d times.\n", h_target, h_count);

    cudaFree(d_sentence);
    cudaFree(d_target);
    cudaFree(d_count);

    return 0;
}

```

OUTPUT]

```
● 6CSED1@debian:~/Desktop/pp1_lab/lab7$ nvcc q1.cu
● 6CSED1@debian:~/Desktop/pp1_lab/lab7$ ./a.out
  The word 'the' appeared 3 times.
○ 6CSED1@debian:~/Desktop/pp1_lab/lab7$
```

Q2] Write a CUDA Program that reads a string S and produces the string RS as follows:
Input S: PCAP Output string RS: PCAPPCAPCP

CODE]

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

__global__ void transformStringKernel(char* S, char* RS, int N) {
    int i = threadIdx.x;
    if (i < N) {

        int offset = 0;
        for (int j = 0; j < i; j++) {
            offset += (N - j);
        }

        int charsToCopy = N - i;
        for (int j = 0; j < charsToCopy; j++) {
            RS[offset + j] = S[j];
        }
    }
}

int main() {
    char h_S[] = "PCAP";
    int N = strlen(h_S);
    int RS_len = (N * (N + 1)) / 2;

    char *d_S, *d_RS;
    char h_RS[RS_len + 1];
```

```

    cudaMalloc((void**)&d_S, N * sizeof(char));
    cudaMalloc((void**)&d_RS, RS_len * sizeof(char));

    cudaMemcpy(d_S, h_S, N * sizeof(char), cudaMemcpyHostToDevice);

    transformStringKernel<<<1, N>>>(d_S, d_RS, N);

    cudaMemcpy(h_RS, d_RS, RS_len * sizeof(char), cudaMemcpyDeviceToHost);
    h_RS[RS_len] = '\0';

    printf("Input string S: %s\n", h_S);
    printf("Output string RS: %s\n", h_RS);

    cudaFree(d_S);
    cudaFree(d_RS);
    return 0;
}

```

OUTPUT]

```

6CSED1@debian:~/Desktop/pp1_lab/lab7$ nvcc q2.cu
6CSED1@debian:~/Desktop/pp1_lab/lab7$ ./a.out
Input string S: PCAP
Output string RS: PCAPPCAPCP
6CSED1@debian:~/Desktop/pp1_lab/lab7$

```

ADDITIONAL QUESTIONS

AQ1] Write a CUDA program that reads a string consisting of N words and reverse each word of it in parallel.

CODE]

```

#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

```

```

__global__ void reverseWordsKernel(char* d_str, int* d_wordStarts, int*
d_wordLens, int numWords) {
    int wordIdx = blockIdx.x * blockDim.x + threadIdx.x;

    if (wordIdx < numWords) {
        int start = d_wordStarts[wordIdx];
        int len = d_wordLens[wordIdx];

        for (int i = 0; i < len / 2; i++) {
            char temp = d_str[start + i];
            d_str[start + i] = d_str[start + len - 1 - i];
            d_str[start + len - 1 - i] = temp;
        }
    }
}

```

```

int main() {
    const char* h_input = "CUDA programming is powerful and fast";
    int n = strlen(h_input);
    char h_str[100];
    strcpy(h_str, h_input);

    int h_starts[50], h_lens[50], numWords = 0;

    int currentLen = 0;
    for (int i = 0; i <= n; i++) {
        if (h_str[i] == ' ' || h_str[i] == '\0') {
            if (currentLen > 0) {
                h_starts[numWords] = i - currentLen;
                h_lens[numWords] = currentLen;
                numWords++;
            }
            currentLen = 0;
        } else {
            currentLen++;
        }
    }

    char *d_str;

```

```

int *d_starts, *d_lens;

cudaMalloc(&d_str, n + 1);
cudaMalloc(&d_starts, numWords * sizeof(int));
cudaMalloc(&d_lens, numWords * sizeof(int));

cudaMemcpy(d_str, h_str, n + 1, cudaMemcpyHostToDevice);
cudaMemcpy(d_starts, h_starts, numWords * sizeof(int),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_lens, h_lens, numWords * sizeof(int),
cudaMemcpyHostToDevice);

reverseWordsKernel<<<1, numWords>>>>(d_str, d_starts, d_lens, numWords);

cudaMemcpy(h_str, d_str, n + 1, cudaMemcpyDeviceToHost);

printf("Original: %s\n", h_input);
printf("Reversed: %s\n", h_str);

cudaFree(d_str); cudaFree(d_starts); cudaFree(d_lens);
return 0;
}

```

OUTPUT]

```

● 6CSED1@debian:~/Desktop/pp1_lab/lab7$ nvcc aq1.cu
● 6CSED1@debian:~/Desktop/pp1_lab/lab7$ ./a.out
Original: CUDA programming is powerful and fast
Reversed: ADUC gnimmargorp si lufrewop dna tsaf
○ 6CSED1@debian:~/Desktop/pp1_lab/lab7$

```

AQ2] Write a CUDA program that takes a string *Sin* as input and one integer value *N* and produces an output string, *Sout*, in parallel by concatenating input string *Sin*, *N* times as shown below.

Input *Sin*: Hello *N*=3

Output *Sout*: HelloHelloHello

CODE]

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

__global__ void concatKernel(char* d_Sin, char* d_Sout, int len, int N) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;

    if (i < len) {
        char c = d_Sin[i];
        for (int j = 0; j < N; j++) {
            int target_idx = (j * len) + i;
            d_Sout[target_idx] = c;
        }
    }
}

int main() {
    const char* h_Sin = "Hello";
    int N = 3;
    int len = strlen(h_Sin);
    int total_len = len * N;

    char *d_Sin, *d_Sout;
    char *h_Sout = (char*)malloc(total_len + 1);

    cudaMalloc((void**)&d_Sin, len);
    cudaMalloc((void**)&d_Sout, total_len);

    cudaMemcpy(d_Sin, h_Sin, len, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (len + threadsPerBlock - 1) / threadsPerBlock;
    concatKernel<<<blocksPerGrid, threadsPerBlock>>>>(d_Sin, d_Sout, len,
N);

    cudaMemcpy(h_Sout, d_Sout, total_len, cudaMemcpyDeviceToHost);
    h_Sout[total_len] = '\0';

    printf("Input: Sin = \"%s\"   N = %d\n", h_Sin, N);
```

```

printf("Output: Sout = \"%s\\n\", h_Sout);

cudaFree(d_Sin);
cudaFree(d_Sout);
free(h_Sout);

return 0;
}

```

OUTPUT]

```

● 6CSED1@debian:~/Desktop/pp1_lab/lab7$ nvcc aq2.cu
● 6CSED1@debian:~/Desktop/pp1_lab/lab7$ ./a.out
Input: Sin = "Hello"  N = 3
Output: Sout = "HelloHelloHello"
○ 6CSED1@debian:~/Desktop/pp1_lab/lab7$ █

```

AQ3] Write a CUDA program which reads a string Sin and produces an output string T as shown below.

Input Sin: Hai

Output T Haaiii

CODE]

```

#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

__global__ void transformKernel(char* Sin, char* T, int n) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;

    if (i < n) {
        char c = Sin[i];
        int numRepeats = i + 1;

        int offset = (i * (i + 1)) / 2;

        for (int j = 0; j < numRepeats; j++) {

```

```

        T[offset + j] = c;
    }
}

int main() {
    const char* h_Sin = "Hai";
    int n = strlen(h_Sin);
    int t_len = (n * (n + 1)) / 2;

    char *d_Sin, *d_T;
    char* h_T = (char*)malloc(t_len + 1);

    cudaMalloc(&d_Sin, n);
    cudaMalloc(&d_T, t_len);

    cudaMemcpy(d_Sin, h_Sin, n, cudaMemcpyHostToDevice);

    transformKernel<<<1, n>>>(d_Sin, d_T, n);

    cudaMemcpy(h_T, d_T, t_len, cudaMemcpyDeviceToHost);
    h_T[t_len] = '\0';

    printf("Input: %s\nOutput: %s\n", h_Sin, h_T);

    cudaFree(d_Sin); cudaFree(d_T); free(h_T);
    return 0;
}

```

OUTPUT]

```

6CSED1@debian:~/Desktop/pp1_lab/lab7$ nvcc aq3.cu
6CSED1@debian:~/Desktop/pp1_lab/lab7$ ./a.out
Input: Hai
Output: Haaii
6CSED1@debian:~/Desktop/pp1_lab/lab7$

```